

Clase 168 — Evasión de defensas: antivirus y EDR

Parte: 7 — Red Team y operaciones ofensivas · Fuente: RTFM v2 (Clark) / documentación de EDR y Windows internals 🕒 Duración estimada: 120 min · Nivel: Experto

Objetivo

Entender cómo funcionan los antivirus y EDR modernos para poder evadirlos de forma comprendida (no por copiar-pegar). El alumno estudiará las técnicas de detección (firmas, heurística, hooks de usermode, ETW, callbacks del kernel) y las contramedidas ofensivas responsables (unhooking, syscalls directas, ejecución en memoria), siempre en su laboratorio y con la mirada puesta en cómo el Blue Team lo detecta.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Describir** cómo un EDR obtiene telemetría (hooks usermode, ETW, callbacks del kernel).
- 2. **Diferenciar** detección estática, heurística y comportamental.
- 3. **Aplicar** técnicas de evasión (unhooking, syscalls indirectas, ejecución en memoria) en un lab.
- 4. **Medir** la evasión frente a un EDR real de laboratorio.
- 5. **Explicar** las contramedidas defensivas de cada técnica ofensiva.

Temas

#	Tema	Por qué importa
1	Firmas y heurística	Detección estática básica
2	Userland hooking	Cómo el EDR intercepta APIs
3	ETW y telemetría	Fuente rica de eventos
4	Kernel callbacks	Visibilidad profunda del EDR
5	Unhooking	Restaurar ntdll limpia
6	Syscalls directas/indirectas	Saltarse los hooks usermode
7	Ejecución en memoria	Evitar tocar disco

Definiciones y características

- **Detección por firma:** comparación con hashes/patrones conocidos. Característica: rápida pero evadible con cambios mínimos.
- **Detección heurística/comportamental:** analiza acciones (inyección, acceso a LSASS). Característica: más robusta, base de los EDR.
- **Userland hooking:** el EDR reescribe funciones de `ntdll.dll` para inspeccionarlas. Característica: evadible con unhooking o syscalls.

- **ETW (Event Tracing for Windows):** telemetría del sistema (ej. `Microsoft-Windows-Threat-Intelligence`). Característica: fuente clave; algunos ataques buscan silenciarla.
- **Kernel callbacks:** notificaciones del kernel (process/thread/image load) que el driver del EDR recibe. Característica: difíciles de evadir desde usermode.
- **Direct/indirect syscalls:** invocar servicios del kernel sin pasar por la `ntdll` hookeada. Característica: evade hooks usermode pero deja otras señales.

Herramientas y preparación

- Un EDR de laboratorio (versión de evaluación o Microsoft Defender for Endpoint en un tenant de prueba) y **Sysmon** (Parte 8) para telemetría.
- Windows internals: entender PE, ntdll y syscalls (repaso Partes 5 y 6).
- Proyectos de estudio open source (SysWhispers y referencias de unhooking) para comprender la mecánica.
- El C2 de las clases anteriores como carga a evadir.

 **Solo laboratorio.** La evasión se practica contra EDR/antivirus en máquinas que controlas, para entender su funcionamiento y mejorar la detección. Distribuir malware evasivo o usarlo fuera de un engagement autorizado es ilegal. El objetivo pedagógico es defensivo: saber qué buscar.

Laboratorio guiado

1. **Línea base de detección.** Ejecuta un implante C2 sin evasión en la VM con EDR y observa la alerta; anota qué evento la disparó.
2. **Analiza los hooks.** Con un depurador, inspecciona una función como `NtAllocateVirtualMemory` en un proceso vigilado y localiza el salto (`jmp`) que el EDR insertó al inicio.
3. **Unhooking conceptual.** Estudia cómo recargar una copia limpia de `ntdll.dll` desde disco (o `KnownDLLs`) para sobrescribir los hooks en memoria; explica por qué restaura las funciones originales.
4. **Syscalls indirectas.** Revisa el patrón de SysWhispers: resolver el número de syscall en runtime y saltar a la instrucción `syscall` dentro de ntdll para no ser hookeado en usermode.
5. **Ejecución en memoria.** Configura tu implante para ejecutarse sin tocar disco (reflective loading) y compara la telemetría frente al binario en disco.
6. **Silenciar ETW (estudio).** Comprende el patcheo de `EtwEventWrite` y por qué reduce visibilidad; observa que el ETW-TI del kernel sigue viendo mucho.
7. **Mide y documenta.** Repite la ejecución con cada técnica y anota si el EDR alerta, degradando el ruido paso a paso, y qué fuente de datos aún lo detecta.

Ejercicios

1. Explica la diferencia entre detección por firma y comportamental con un ejemplo.
2. Describe cómo un EDR instala un hook en ntdll y cómo el unhooking lo revierte.
3. Compara syscalls directas e indirectas en términos de sigilo.
4. Explica por qué la ejecución en memoria genera menos telemetría de disco.
5. Investiga qué es ETW-TI y por qué es más difícil de evadir que ETW clásico.
6. Para tres técnicas ofensivas, describe su contramedida defensiva.

Reto verificable

Partiendo de un implante que el EDR de tu lab detecta, aplica **al menos dos técnicas de evasión** y documenta el cambio en la telemetría. **Criterio de aceptación:** demuestras (con capturas de la consola del EDR/Sysmon) que la línea base generaba una alerta y que tras las técnicas aplicadas cambia el resultado; además, explicas qué fuente de datos aún podría detectarte. Todo en tu laboratorio.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
Evasión "mágica" que no entiendes	Copiaste código; estudia el mecanismo antes de usarlo
Sigue detectando pese a syscalls	El EDR usa kernel callbacks/ETW-TI; usermode no basta
El unhooking crashea el proceso	Copia de ntdll mal mapeada; respeta secciones/permisos
Alerta por acceso a LSASS	La técnica es comportamental, no de firma; cambia el comportamiento, no el binario
Funciona hoy, falla mañana	El EDR se actualizó; la evasión es una carrera continua

Preguntas frecuentes

 **¿Cambiar el hash evade un EDR?** Solo evade firmas estáticas. Los EDR modernos son comportamentales: detectan lo que el proceso hace, no cómo se llama el archivo.

 **¿Las syscalls directas son la solución definitiva?** No. Evaden hooks usermode, pero el kernel (callbacks, ETW-TI) sigue viendo la actividad. Además, un proceso que llama syscalls "raras" es en sí sospechoso.

 **¿Esto no es enseñar a hacer malware?** Enseñamos el mecanismo para **defender**: sin entender la evasión no se puede escribir una detección robusta. Todo se practica en laboratorio propio.

Referencias

- MITRE ATT&CK — *Defense Evasion* (TA0005). <https://attack.mitre.org/tactics/TA0005/>
- Elastic Security Labs / research sobre EDR internals y hooking.
- SysWhispers (estudio de syscalls). <https://github.com/klezVirus/SysWhispers3>
- Clark, B. — *RTFM: Red Team Field Manual v2*.

Siguiente clase

Clase 169 - Ofuscacion de payloads y bypass de AMSI